

Good Numbers Counting

This is a very nice **combinatorics + modular inverse** problem. The key is to realize that we **don't need to generate any of the length- n numbers**.

1. First understand what a "good" number is

We have two digits:

a, b

Every good number contains **only a and b** .

For example:

$a = 1, b = 3$

Good:

1
3
13
31
113
333

Not good:

2
12
123

We need to count length n numbers whose **digit sum is also good**.

2. The important observation

Suppose our length n number contains:

- k digits equal to b
- $n-k$ digits equal to a

Then its digit sum is:

$$(n - k)a + kb$$

or equivalently:

$$na + k(b - a)$$

For example:

$a = 1$
 $b = 3$
 $n = 3$

Consider numbers having:

0 occurrences of 3

111

Sum:

$$1 + 1 + 1 = 3$$

3 contains only digits $\{1, 3\}$, so it is good.

1 occurrence of 3

Numbers:

113
131
311

Sum:

$$1 + 1 + 3 = 5$$

5 is not good.

2 occurrences of 3

133
313
331

Sum:

7

Not good.

3 occurrences of 3

333

Sum:

9

Not good.

Therefore only $k = 0$ works.

But there is another important question:

If a particular k works, how many numbers have exactly k occurrences of b ?

That's simply:

$$\binom{n}{k}$$

because we choose which k positions contain b .

So the answer becomes:

$$\sum_{\substack{0 \leq k \leq n \\ (n-k)a+kb \text{ is good}}} \binom{n}{k}$$

This is the entire idea.

3. Why $c(n, k)$?

Suppose:

$n = 5$
 $k = 2$

We need exactly two b s.

For example:

a a b b a
a b a b a
b a a b a
b b a a a
...

We simply choose the 2 positions where b will go.

Number of ways:

$$\binom{5}{2} = 10$$

So every valid digit sum contributes $c(n, k)$ answers.

4. How do we check whether the sum is good?

For every:

```
k = 0 ... n
```

calculate:

```
C++
sum = (n-k)*a + k*b;
```

Now check whether every digit of `sum` is either `a` or `b`.

For example:

```
a = 1, b = 3
sum = 313
```

Check:

```
3 -> good
1 -> good
3 -> good
```

So `313` is good.

But:

```
sum = 315
```

contains `5`, so it isn't good.

5. Why can we check only $n + 1$ possibilities?

This is the beautiful part.

There are:

$$2^n$$

possible good numbers of length n .

For $n = 10^6$, impossible to enumerate.

But their digit sums depend **only on how many bs they contain**.

There are only:

$$n + 1$$

possible values of k .

So instead of checking:

```
2^n numbers
```

we check:

```
n + 1 numbers
```

That's the main intuition of the problem.

6. Computing $c(n, k)$ efficiently

We need:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Since the answer must be modulo:

$$MOD = 10^9 + 7$$

we can't directly divide.

We use Fermat's Little Theorem:

$$x^{-1} \equiv x^{MOD-2} \pmod{MOD}$$

Therefore:

$$\binom{n}{k} = n! \times (k!)^{-1} \times ((n-k)!)^{-1}$$

So precompute:

```
C++
fact[i] = i!
```

and

```
C++
invFact[i] = (i!)^-1
```

Then:

```
C++
C(n,k) = fact[n] * invFact[k] * invFact[n-k]
```

in $O(1)$.

7. Complexity

We have $n + 1$ possible values of k .

For each k :

1. Calculate sum $\rightarrow O(1)$
2. Check its digits $\rightarrow$ at most about 7 digits because $n \leq 10^6$ and $a, b \leq 9$
3. Calculate combination $\rightarrow O(1)$

Factorials take $O(n)$.

So:

$$O(n \log MOD)$$

because of modular exponentiation for the inverse factorial.

Practically this is easily fast enough for $n = 10^6$.

Memory:

$$O(n)$$

8. C++ Code

```
C++
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

const ll MOD = 1000000007LL;

// Fast power: a^b % MOD
ll modPow(ll a, ll b) {
    ll res = 1;

    while (b > 0) {
        if (b & 1)
            res = res * a % MOD;

        a = a * a % MOD;
        b >>= 1;
    }

    return res;
}

// Check whether x contains only digits a and b
bool isGood(ll x, int a, int b) {
    while (x > 0) {
        int digit = x % 10;

        if (digit != a && digit != b)
            return false;

        x /= 10;
    }

    return true;
}
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int a, b, n;
    cin >> a >> b >> n;

    // factorials
    vector<ll> fact(n + 1);
    vector<ll> invFact(n + 1);

    fact[0] = 1;

    for (int i = 1; i <= n; i++) {
        fact[i] = fact[i - 1] * i % MOD;
    }

    // inverse factorial
    invFact[n] = modPow(fact[n], MOD - 2);
    for (int i = n; i >= 1; i--) {
        invFact[i - 1] = invFact[i] * i % MOD;
    }

    // C(n, k)
    auto nCr = [&](int n, int k) -> ll {
        return fact[n] * invFact[k] % MOD
            * invFact[n - k] % MOD;
    };

    ll ans = 0;
    for (int k = 0; k <= n; k++) {
        // k digits are b
        // n-k digits are a
        ll sum = 1LL * (n - k) * a + 1LL * k * b;

        // Is digit sum a good number?
        if (isGood(sum, a, b)) {
            ans = (ans + nCr(n, k)) % MOD;
        }
    }

    cout << ans << '\n';
    return 0;
}
```

9. Dry run

Take:

```
a = 1
b = 3
n = 3
```

We examine every possible k.

k	# of 3s	# of 1s	Sum	Good?	Ways
0	0	3	3	✓	C(3,0)=1
1	1	2	5	✗	C(3,1)=3
2	2	1	7	✗	C(3,2)=3
3	3	0	9	✗	C(3,3)=1

Only k=0 is valid.

Therefore:

$$answer = 1$$

which matches the example.

10. The intuition you should remember

When you see this problem, **don't think about individual strings**.

Instead think:

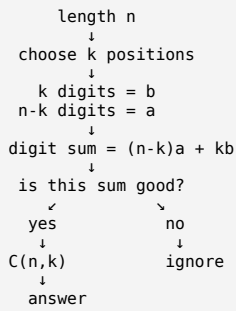
What information about the string actually affects the condition?

The condition is based on the **sum of digits**.

Since there are only two possible digits, the sum is completely determined by:

How many times did I choose b?

So:



That's the core pattern:

Group all strings by the number of occurrences of one digit

This converts an exponential problem 2^n into a linear problem n .

One important edge case: `sum` is always positive because `a >= 1`, so `isGood()` never receives `0`.

Today 9:43 AM

If you're asking **what exactly the combination $C(n, k)$ represents in this problem**, it's:

$C(n, k)$ = number of ways to place exactly k copies of b

among the n positions.

Example

Suppose:

```

a = 1
b = 3
n = 4

```

Take `k = 2`.

That means we want exactly **two 3s** and therefore two **1s**.

Possible numbers include:

```

3311
3131
3113
1331
1313
1133

```

There are **6** of them.

And:

$$C(4, 2) = \frac{4!}{2!2!} = 6$$

Why not multiply by something else?

Because once you choose the positions of the `bs`, **all remaining positions automatically contain `a`**.

For example, choosing positions `{1, 3}` gives:

```

3 1 3 1

```

So we only need to choose:

Which `k` positions contain `b`?

That's exactly:

$C(n, k)$

In this problem

For every `k`:

```

C++
sum = (n-k)*a + k*b;

```

If **sum** is good, then **all** $C(n, k)$ **numbers having exactly k b's are excellent.**

So:

$$answer = \sum_{\text{valid } k} C(n, k)$$

For example, if $n = 5$ and valid k values are **0, 2, 4**:

$$\begin{aligned} answer &= C(5, 0) + C(5, 2) + C(5, 4) \\ &= 1 + 10 + 5 = 16 \end{aligned}$$

That's why combination appears here.

Exactly — you're asking the **most important subtle point** here.

You are right:

In combinations, order doesn't matter.

So why are we using $C(n, k)$ when **3311** and **3131** are **different numbers**?

The answer is: **we are not using combination to represent the final number. We use combination to choose the positions of b.**

Think about positions, not the numbers

Suppose:

```
n = 4
a = 1
b = 3
k = 2
```

We need:

exactly **2 positions containing 3.**

Our 4 positions are:

```
Position:  1  2  3  4
           -  -  -  -
```

We choose **2 positions out of 4** for **3**.

That's:

$$C(4, 2) = 6$$

Now look at each selection.

Selection 1

Choose positions:

```
{1, 2}
```

Put **3** there, and **1** everywhere else:

```
3 3 1 1
```

→ **3311**

Selection 2

Choose:

```
{1, 3}
```

Gives:

```
3 1 3 1
```

→ **3131**

Selection 3

Choose:

{1, 4}

Gives:

3 1 1 3

→ 3113

Selection 4

Choose:

{2, 3}

Gives:

1 3 3 1

→ 1331

Selection 5

Choose:

{2, 4}

Gives:

1 3 1 3

→ 1313

Selection 6

Choose:

{3, 4}

Gives:

1 1 3 3

→ 1133

So:

$$C(4, 2) = 6$$

and the **6 different selections of positions produce 6 different numbers.**

The key distinction

You said:

"order doesn't matter"

Correct.

But **the order of the positions is not what we're choosing.**

We're choosing a **set of positions.**

For example:

{1, 3}

and

{3, 1}

are the **same combination**.

Both mean:

Put **3** at position 1 and position 3.

And they produce the same number:

3131

But:

{1, 3}

and

{2, 3}

are different combinations.

They produce:

3131
1331

which are different numbers.

This is the beautiful trick

We are **not counting permutations of 1, 1, 3, 3**.

We're doing:

Choose positions for 3
↓
Remaining positions automatically become 1
↓
Each position selection creates exactly ONE number

Therefore:

$$C(n, k)$$

is exactly the number of distinct length-**n** numbers containing **k** copies of **b**.

Compare with permutation

If you asked:

How many different arrangements of **1, 1, 3, 3** exist?

Then you'd use:

$$\frac{4!}{2!2!} = 6$$

Interestingly, you get the same answer.

And that's because:

$$\frac{4!}{2!2!} = C(4, 2)$$

So in this specific situation, **choosing positions for the bs** and **arranging repeated digits** are two equivalent ways of looking at the same counting problem.

The **position-selection viewpoint is much easier to use in this problem**.

ChatGPT can make mistakes. Check important info.